

# Documentação do MVPFy

Há dois percursos documentais:

## Percurso

[Guia do usuário](#)

[Guia de desenvolvimento](#)

## Uso

Instalação, entrevista e geração do plano.

Arquitetura, contratos, scripts e testes.

Os dois percursos e a especificação são compilados para PDF e EPUB conforme `reading-order.txt`.

## Guia do usuário

O MVPFy conduz uma conversa curta para estruturar uma empresa em formação e seu primeiro SaaS. Você não precisa conhecer termos de produto, marketing ou infraestrutura: a pergunta aparece com linguagem simples e a análise acontece nas especialistas.

Comece por [instalação](#), avance para o [primeiro MVP](#) e consulte a explicação da [entrevista](#) quando quiser entender a retomada.

## Instalação

No projeto consumidor, instale o catálogo:

```
npx skills add promovaweb/mvpfy
```

Depois, peça ao agente: “Quero começar meu primeiro MVP SaaS com o MVPFy”. A skill cria ou usa `.mvpfy/` e mantém o plano em `Company.md`.

Para preparar a estrutura sem iniciar a conversa:

```
node skills/mvpfy/scripts/setup-project.mjs --project .
```

Não coloque respostas de uma empresa cliente no repositório da biblioteca.

## Seu primeiro MVP SaaS

O primeiro encontro começa pela situação problemática, não pelo nome da tela. O agente pergunta quem sente a dificuldade, qual alternativa existe hoje e qual resultado precisa funcionar do começo ao fim.

Depois, o plano conecta público, jornada, conta, onboarding, cobrança, suporte, preço, custo, tecnologia, marketing e métricas. O escopo posterior fica separado para manter a versão 1.0 pequena o bastante para aprender com os primeiros clientes.

Quando fizer sentido, o plano recomenda Laravel em uma VPS, banco PostgreSQL, armazenamento de arquivos, e-mail transacional e um provedor de IA apenas para uma função clara do produto.

## Entrevista e retomada

O MVPFy lê o estado antes de cada turno. Uma resposta pode preencher várias áreas e a próxima pergunta muda conforme o que já foi informado.

O fluxo é:

1. receber a resposta;
2. registrar o texto original e sua interpretação;
3. atualizar o estado e as seções afetadas;
4. confirmar a gravação;
5. apresentar uma única pergunta nova.

Você pode responder com o número de uma opção, escrever livremente, dizer que não sabe, corrigir algo, pausar ou pedir uma área específica. Ao voltar, diga “quero continuar meu MVPFy”. O agente retoma pela pendência mais importante.

## O arquivo Company.md

O Company.md é a entrega central. Ele não é uma transcrição: organiza o que foi informado, o que foi recomendado, as hipóteses e as perguntas ainda abertas.

As seções cobrem resumo, empresa, problema, público, personas, marca, jornada, conta, onboarding, escopo, módulos, assinatura, suporte, mercado, preço, economia, tecnologia, operação, IA, website, marketing, vendas, métricas, ameaças, execução, escolhas, hipóteses e fontes.

Uma versão parcial usa `document_status`: `preliminary`. O estado `ready` só aparece após a presença dos campos mínimos do primeiro SaaS.

# Solução de problemas

## A entrevista voltou ao começo

Confira se `.mvpfy/state.json`, `.mvpfy/answers.jsonl` e `Company.md` estão no mesmo projeto. O agente reconstrói o estado a partir do `Company.md` quando os arquivos auxiliares não estão disponíveis.

## A pergunta não avançou

O MVPFy só avança depois da gravação. Verifique a mensagem do script `record-answer.mjs` e tente responder novamente sem apagar os arquivos do projeto.

## O template mudou

Use `$mvpfy-migrate`. A skill adiciona blocos ausentes, mantém o conteúdo preenchido e apresenta apenas a próxima pergunta nova relevante.

# Guia de desenvolvimento

O repositório distribui skills independentes e scripts determinísticos. A orquestradora coordena a conversa, enquanto as especialistas mantêm o método de seus domínios.

Leia [arquitetura](#), [contratos](#), [artefatos](#) e [testes](#) antes de alterar o pacote.

## Arquitetura

Cada skill vive em `skills/mvpfy-*` e pode ser instalada separadamente. O projeto consumidor guarda o processo em `.mvpfy/`, com estado mutável, eventos append-only, pesquisas e versão do template. O resultado é `Company.md`.

`mvpfy` lê o pedido, escolhe a especialista e garante a ordem da conversa. `mvpfy-document` controla a estrutura. `mvpfy-migrate` evolui o template. As outras skills retornam campos, recomendações, lacunas e uma pergunta opcional.

## Contratos

Especialistas recebem um pacote YAML com projeto, pedido, campos conhecidos, escolhas, hipóteses, lacunas, conflitos e modo da entrevista. A

resposta retorna fatos, escolhas, recomendações, campos resolvidos, lacunas e `next_question`.

A especialista não deve perguntar várias coisas nem escrever fora da própria área. A orquestradora integra retornos e elimina repetição.

## Artefatos

| Arquivo                              | Função                              |
|--------------------------------------|-------------------------------------|
| <code>Company.md</code>              | Plano único entregue ao usuário.    |
| <code>.mvpfy/state.json</code>       | Estado atual e fila de trabalho.    |
| <code>.mvpfy/answers.jsonl</code>    | Histórico append-only de respostas. |
| <code>.mvpfy/research.json</code>    | Referências de mercado consultadas. |
| <code>.mvpfy/template-version</code> | Versão usada na última migração.    |

Os IDs `mvpfy:section:<id>` tornam a atualização idempotente e permitem que o template evolua sem substituir texto já preenchido.

## Testes

Execute:

```
npm test
npm run ebook:verify
```

Os testes conferem as 12 skills, frontmatter, metadados de interface, IDs do template e os scripts de setup, registro, renderização e migração. Teste manual recomendado: iniciar um projeto temporário, registrar uma resposta, executar a migração duas vezes e conferir que nenhum bloco foi duplicado.

## Contribuição

Altere a skill especialista quando o novo conhecimento pertencer a um domínio. Altere a orquestradora somente para regras de fluxo, retomada e integração. Mudanças no template exigem atualização de `ebooks/VERSION`, testes de migração e novo build do ebook.

Antes de abrir uma alteração, rode `npm test`, leia o `Company.md` de exemplo e execute `npm run ebook:verify` quando a documentação tiver mudado.

# Especificação do MVPFy

## Objetivo

Produzir um plano detalhado para uma empresa em formação que vai criar seu primeiro software como serviço. O recorte é a versão 1.0, não o produto completo.

## Entrevista

Uma pergunta por turno, salvamento antes da próxima, retomada por estado e aproveitamento semântico de cada resposta. O padrão é de 10 a 14 perguntas centrais, com aprofundamento apenas quando problema, público, jornada, preço ou operação ainda estiverem indefinidos.

## Especialistas

Problema, público, produto, SaaS, marca, mercado, preço, tecnologia e marketing mantêm o conhecimento dos próprios domínios. Documento e migração cuidam do artefato final. A orquestradora conecta os retornos.

## Entrega

Company.md precisa orientar produto, desenvolvimento, website, marca, marketing, vendas e operação. Ele separa informação do usuário, recomendação, hipótese, comprovação e pendência.

## Tecnologia preferencial

Laravel é a base recomendada, com VPS, banco PostgreSQL, arquivos, e-mail e IA somente quando houver função clara. Node.js entra com justificativa concreta.

## Migração

O template tem versão semântica e IDs estáveis. Novos blocos entram na posição canônica, sem apagar o conteúdo existente. Após a migração, a entrevista volta apenas para a lacuna nova mais importante.